

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: CSA14

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO3: *Analyze syntax-directed translation method using synthesized and inherited attributes to generate a Parser Tree. (BL4)*

Assessment Tool Weightage: 10%

QUESTIONS: 50

Annexure A

**Duration :60 Mins
On Class
Total Marks:50**

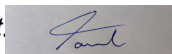
SCENARIO BASED TASK

Code of Conduct:

I Tamil Elakiya S (192524443) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student.



Annexure A

SCENARIO BASED TASK

Q1. A compiler development team is implementing a simple desk-calculator module that evaluates arithmetic expressions entered by users. The calculator supports addition, subtraction, multiplication, division, parentheses, and single-digit operands. The compiler uses a Syntax-Directed Definition (SDD) to compute the value of each expression during parsing.

The grammar used by the calculator is:

$$S \rightarrow E n$$
$$E \rightarrow E + T$$
$$| E - T$$
$$| T$$
$$T \rightarrow T * F$$
$$| T / F$$
$$| F$$
$$F \rightarrow (E)$$
$$| \text{digit}$$

For the input expression:

$$(6 + 4) * (4 + 3)$$

Analyze the grammar and design an appropriate SDD for evaluating arithmetic expressions. Construct the annotated parse tree for the given input and show the attribute values computed at each relevant node. Finally, determine the final value produced by the calculator.

Q2. A compiler development team is testing the syntax-directed translation module of an arithmetic expression processor. During testing, the expression

$$a * b - c + 8$$

is given as input. The compiler must correctly represent the expression according to operator precedence and associativity and use a Syntax-Directed Definition (SDD) to evaluate the expression.

Analyze the given expression, construct its syntax tree considering the precedence and associativity of the operators, and formulate an appropriate SDD for evaluating the expression. Show the attribute evaluation sequence for the given expression.

Q3. A compiler development team is designing a semantic-analysis module that uses inherited and synthesized attributes to pass information between grammar symbols during translation. The team needs to verify whether the semantic rules used in the grammar satisfy the conditions of an L-attributed definition, so that the attributes can be evaluated from left to right during parsing.

Consider the production rules:

$A \rightarrow P Q$

$A \rightarrow X Y$

Each of the non-terminals A, P, Q, X, and Y has two attributes:

- s – synthesized attribute
- i – inherited attribute

The compiler team proposes the following semantic rules:

Rule 1:

$P.i = A.i + 2$

$Q.i = P.i + A.i$

$A.s = P.s + Q.s$

Rule 2:

$X.i = A.i + Y.s$

$Y.i = X.s + A.i$

Analyze the two sets of semantic rules with respect to the conditions of an L-attributed definition. Determine which rule satisfies the requirements of an L-attributed definition and which rule violates them. Justify your answer by examining the dependency of inherited and synthesized attributes and their left-to-right evaluation order.

Q4. A compiler development team is implementing an expression translator for a calculator application. The translator must evaluate arithmetic expressions and generate an appropriate intermediate code representation using an S-attributed Syntax-Directed Definition (SDD). The syntax analyzer uses LR parsing to process the input from left to right and construct the required parse structure.

The compiler uses the following grammar:

$S \rightarrow E N$

$E \rightarrow E + T$

$E \rightarrow E - T$

$E \rightarrow T$

$T \rightarrow T * F$

$T \rightarrow T / F$

$T \rightarrow F$

$F \rightarrow (E)$

$F \rightarrow \text{digit}$

$N \rightarrow ;$

The input expression provided to the translator is:

$2 + 5 * 6 ;$

Analyze the given grammar and design an S-attributed Syntax-Directed Definition for generating the corresponding code fragment (translator). Then trace the LR parsing process for the input string $2 + 5 * 6 ;$ using a parser stack, showing the sequence of shift and reduce operations. Finally, show the generated translation for the given input.

Q5. A compiler development team is implementing a top-down expression translator for an arithmetic calculator. The translator must process arithmetic expressions containing addition, multiplication, parentheses, and single-digit operands. During translation, the compiler should use a Syntax-Directed Definition (SDD) to compute the value of the expression and annotate the corresponding parse tree.

The grammar used by the translator is:

$S \rightarrow E n$

$E \rightarrow E + T$
 $| T$

$T \rightarrow T * F$
 $| F$

$F \rightarrow (E)$
 $| \text{digit}$

The compiler is required to process the following input:

$3 * 4 + 8 n$

Analyze the given grammar and formulate a suitable SDD for top-down translation. Construct the annotated parse tree for the given input expression, showing the attribute values at the appropriate nodes. Finally, trace the top-down evaluation and determine the final value generated by the translator.

RUBRICS FOR CALCULATION

Scenario based assessment

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts

Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Name: Tamil Elakiyas

Reg No: 192524443

Course: Compiler Design

Course

Code: CSA1408

Co-3

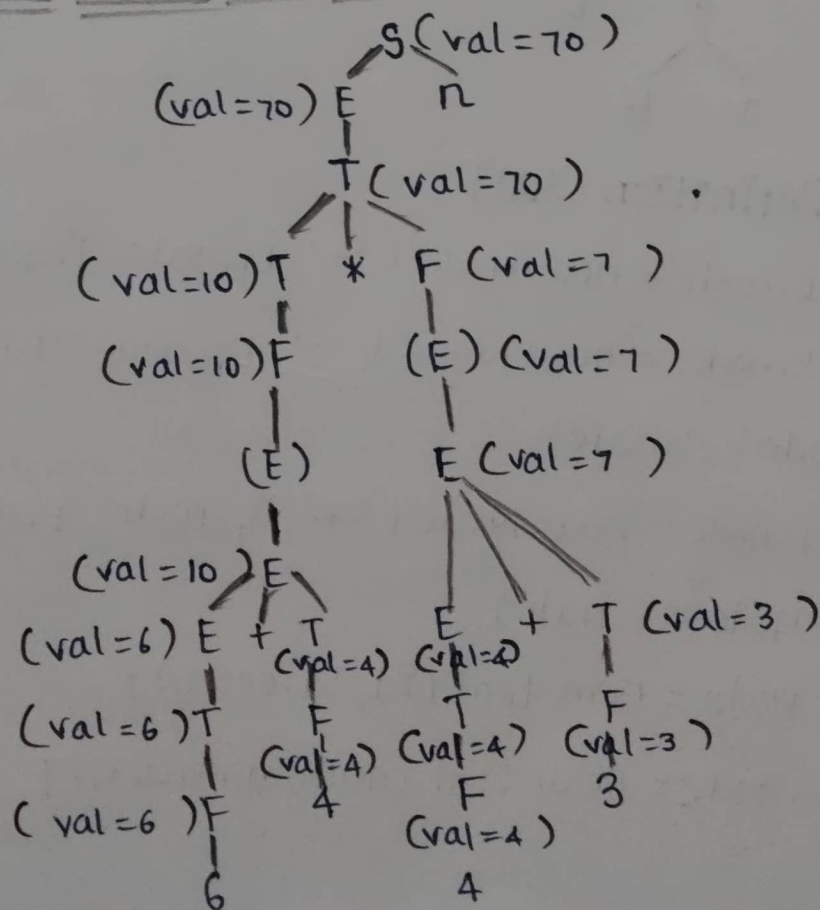
Assessment tool-3

Scenario Based

Q1. Syntax-Directed Definition (SDD):

- * $S \rightarrow E_n \{S.val = E.val\}$
- * $E \rightarrow E_1 + T \{E.val = E_1.val + T.val\}$
- * $E \rightarrow E_1 - T \{E.val = E_1.val - T.val\}$
- * $E \rightarrow T \{E.val = T.val\}$
- * $T \rightarrow T_1 * F \{T.val = T_1.val * F.val\}$
- * $T \rightarrow T_1 / F \{T.val = T_1.val / F.val\}$
- * $T \rightarrow F \{T.val = F.val\}$
- * $F \rightarrow (E) \{F.val = E.val\}$
- * $F \rightarrow \text{digit} \{F.val = \text{digit.lexval}\}$

Annotated Parse Tree for $(6+4) * (4+3)$



Left Subtree: (6+4):

$F \rightarrow \text{digit}(6) \Rightarrow F.\text{val} = 6$

$T \rightarrow F \Rightarrow T.\text{val} = 6$

$E \rightarrow T \Rightarrow E.\text{val} = 6$

$F \rightarrow \text{digit}(4) \Rightarrow F.\text{val} = 4$

$T \rightarrow F \Rightarrow T.\text{val} = 4$

$E \rightarrow E(6) + T(4) \Rightarrow E.\text{val} = 10$

$F \rightarrow (E(10)) \Rightarrow F.\text{val} = 10$

$T \rightarrow F \Rightarrow T.\text{val} = 10$

Right Subtree: (4+3)

$F \rightarrow \text{digit}(4) \Rightarrow F.\text{val} = 4$

$T \rightarrow F \Rightarrow T.\text{val} = 4$

$E \rightarrow T \Rightarrow E.\text{val} = 4$

$F \rightarrow \text{digit}(3) \Rightarrow F.\text{val} = 3$

$T \rightarrow F \Rightarrow T.\text{val} = 3$

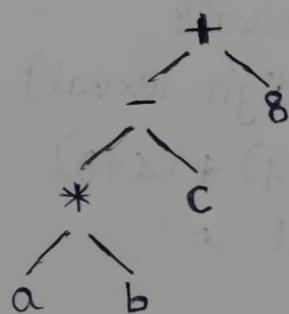
$E \rightarrow E(4) + T(3) \Rightarrow E.\text{val} = 7$

$F \rightarrow (E(7)) \Rightarrow F.\text{val} = 7$

Final Computed value : 70

Q2. Syntax Tree for: $a * b - c + 8$

Considering standard operator precedence ($* > - = +$) and left-to-right associativity:



Syntax tree

Syntax-Directed Definition (SDD)

- $E \rightarrow E_1 + T \quad \{E.\text{node} = \text{new Node}(' + ', E_1.\text{node}, T.\text{node})\}$
- $E \rightarrow E_1 - T \quad \{E.\text{node} = \text{new Node}(' - ', E_1.\text{node}, T.\text{node})\}$
- $E \rightarrow T \quad \{E.\text{node} = T.\text{node}\}$
- $T \rightarrow T_1 * F \quad \{T.\text{node} = \text{new Node}(' * ', T_1.\text{node}, F.\text{node})\}$
- $T \rightarrow F \quad \{T.\text{node} = F.\text{node}\}$
- $F \rightarrow \text{id} \quad \{F.\text{node} = \text{new Leaf}(\text{id}, \text{id}.\text{entry})\}$
- $F \rightarrow \text{num} \quad \{F.\text{node} = \text{new Leaf}(\text{num}, \text{num}.\text{val})\}$

Attribute Evaluation Sequence.

1. $\text{Leaf}_1 = \text{new Leaf}(\text{id}, a)$
2. $\text{Leaf}_2 = \text{new Leaf}(\text{id}, b)$
3. $\text{Node}_1 = \text{new Node}('*', \text{Leaf}_1, \text{Leaf}_2)$
4. $\text{Leaf}_3 = \text{new Leaf}(\text{id}, c)$
5. $\text{Node}_2 = \text{new Node}('-', \text{Node}_1, \text{Leaf}_3)$
6. $\text{Leaf}_4 = \text{new Leaf}(\text{num}, 8)$
7. $\text{Node}_3 = \text{new Node}('+', \text{Node}_2, \text{Leaf}_4)$

Q3. L-Attributed Definition Conditions:

An SDD is L-attributed if each inherited attribute of a symbol on the right-hand side of a production $A \rightarrow Y_1 Y_2 \dots Y_n$ depends only on:

- The inherited attributes of the head non-terminal A .
- The attributes (synthesized or inherited) of symbols $Y_1, Y_2, \dots, Y_{k-1}$ located to the left of Y_k .

Analysis of Rule Sets:

• Rule 1 ($A \rightarrow PQ$):

- $P.i = A.i + 2$:

Inherited attribute of P depends on $A.i$ (Head attribute)
 $\rightarrow$ valid

- $Q.i = P.i + A.i$:

Inherited attribute of Q depends on $P.i$ (symbol to its left) and $A.i$ (Head attribute) $\rightarrow$ valid.

- $A.s = P.s + Q.s$:

Synthesized attribute of A depends on children $P.s$ and $Q.s \rightarrow$ valid.

• Rule 2 ($A \rightarrow XY$):

• $X.i = A.i + Y.s$:

Inherited attribute of X depends on $Y.s$, which is a symbol to its right $\rightarrow$ Violation.

• $Y.i = X.s + A.i$:

Inherited attribute of Y depends on $X.s$ (Symbol to its left) and $A.i$ (Head attribute) $\rightarrow$ valid.

Thereby,

• Satisfies L-attributed definition: Rule 1

• Violates L-attributed definition: Rule 2 (due to the right-to-left dependency)

Q4. S - Attributed Syntax - Directed Definition (SDD)

• $S \rightarrow E$; $\{S.code = E.code$; $\text{print}(S.code); \}$

• $E \rightarrow E_1 + T$ $\{E.code = E_1.code \parallel T.code \parallel "+" ; \}$

• $E \rightarrow E_1 - T$ $\{E.code = E_1.code \parallel T.code \parallel "-" ; \}$

• $E \rightarrow T$ $\{E.code = T.code ; \}$

• $T \rightarrow T_1 * F$ $\{T.code = T_1.code \parallel F.code \parallel "*" ; \}$

• $T \rightarrow T_1 / F$ $\{T.code = T_1.code \parallel F.code \parallel "/" ; \}$

• $T \rightarrow F$ $\{T.code = F.code ; \}$

• $F \rightarrow (E)$ $\{F.code = E.code ; \}$

• $F \rightarrow \text{digit}$ $\{F.code = \text{digit.lexval} ; \}$

LR Parsing and Attribute Evaluation Trace: $(2+5*6;)$

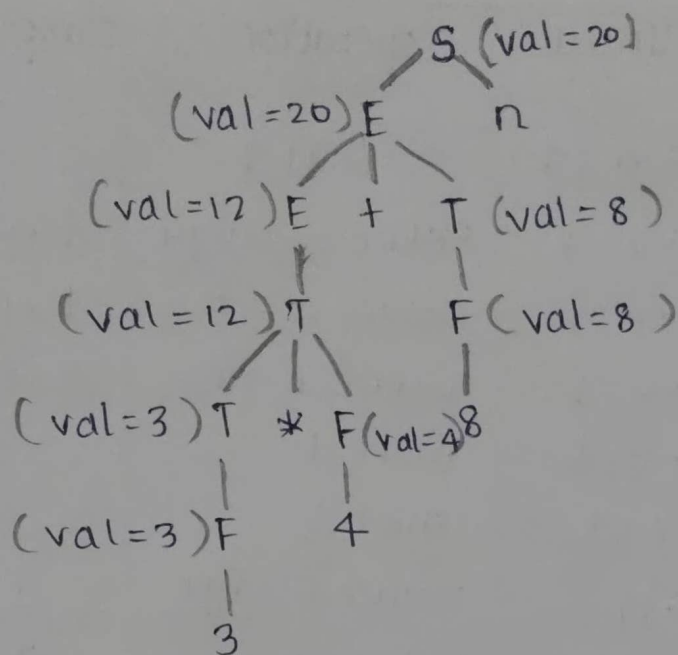
Step	Stack	Input	operation	computed Attribute
1	\$	2+5*6;\$	Shift 2	-
2	\$digit ₂	+5*6;\$	Reduce $F \rightarrow \text{digit}$	$F.\text{code} = '2'$
3	\$F	+5*6;\$	Reduce $T \rightarrow F$	$T.\text{code} = '2'$
4	\$T	+5*6;\$	Reduce $E \rightarrow T$	$E.\text{code} = '2'$
5	\$E	+5*6;\$	Shift +	-
6	\$E+	5*6;\$	Shift 5	-
7	\$E+digit ₅	*6;\$	Reduce $F \rightarrow \text{digit}$	$F.\text{code} = '5'$
8	\$E+F	*6;\$	Reduce $T \rightarrow F$	$T.\text{code} = '5'$
9	\$E+T	*6;\$	Reduce Shift *	-
10	\$E+T*	6;\$	Shift 6	-
11	\$E+T*digit ₆	;\$	Reduce $F \rightarrow \text{digit}$	$F.\text{code} = '6'$
12	\$E+T*F	;\$	Reduce $T \rightarrow T*F$	$T.\text{code} = "56*"$
13	\$E+T	;\$	Reduce $E \rightarrow E+T$	$E.\text{code} = "256*+"$
14	\$E	;\$	Shift ;	-
15	\$E;	\$	$S \rightarrow E;$	$S.\text{code} = "256*+"$
16	\$S	\$	Accept.	Intermediate postfix code generated

Postfix Intermediate Code : 2 5 6 * +

Q5. Syntax - Directed Definition (SDD):

- $S \rightarrow E_n \quad \{S.\text{val} = E.\text{val}\}$
- $E \rightarrow E_1 + T \quad \{E.\text{val} = E_1.\text{val} + T.\text{val}\}$
- $E \rightarrow T \quad \{E.\text{val} = T.\text{val}\}$
- $T \rightarrow T_1 * F \quad \{T.\text{val} = T_1.\text{val} * F.\text{val}\}$
- $T \rightarrow F \quad \{T.\text{val} = F.\text{val}\}$
- $F \rightarrow (E) \quad \{F.\text{val} = E.\text{val}\}$
- $F \rightarrow \text{digit} \quad \{F.\text{val} = \text{digit.lexval}\}.$

Annotated Parse Tree for $3 * 4 + 8n$



Top-Down Evaluation Sequence & Final output:

- 1). Parse $F \rightarrow \text{digit}(3) \Rightarrow F.\text{val} = 3$
- 2). Parse $T \rightarrow F \Rightarrow T.\text{val} = 3$
- 3). Parse $F \rightarrow \text{digit}(4) \Rightarrow F.\text{val} = 4$
- 4). Compute $T \rightarrow T(3) * F(4) \Rightarrow T.\text{val} = 3 \times 4 = 12$
- 5). Compute $E \rightarrow T(12) \Rightarrow E.\text{val} = 12$
- 6). Parse $F \rightarrow \text{digit}(8) \Rightarrow F.\text{val} = 8$
- 7). Compute $T \rightarrow F(8) \Rightarrow T.\text{val} = 8$
- 8). Compute $E \rightarrow E(12) + T(8) \Rightarrow E.\text{val} = 12 + 8 = 20$
- 9). Compute $S \rightarrow E(20)n \Rightarrow S.\text{val} = 20$

Final computed value : 20.